Quantstamp

Nillion | Staking Contracts

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Staking
Timeline	2026-03-11 through 2026-03-23
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• NillionNetwork/blacklight-contracts #8ce4597
Auditors	• Andrei Stefan Auditing Engineer • Andy Lin Senior Auditing Engineer • Yamen Merhi Auditing Engineer

Documentation quality	Medium
Test quality	Medium
Total Findings	10 Fixed: 10
High severity findings	0
Medium severity findings	2 Fixed: 2
Low severity findings	3 Fixed: 3
Undetermined severity findings	0
Informational findings	5 Fixed: 5

Summary of Findings

The Nillion Staking Security Audit has uncovered several vulnerabilities within the contracts, primarily concerning `NodeOperatorFactory` and `NodeOperator`. A medium severity issue is present in the implementation of `stakeTo()`, it can be exploited by an unauthorized user to bypass the Factory's binding mechanism for nodes, allowing front-running and blocking legitimate staking efforts.

Recommendations largely focus on strengthening checks and safeguards. Additionally, introducing functionality for better handling of fee settings, ensuring token allowances are revoked on updates, and implementing thorough documentation of methods would substantially elevate the integrity of the code. Overall, addressing these vulnerabilities and implementing the recommendations will enhance security within the Nillion staking ecosystem, ensuring greater protection against potential exploitation.

Fix Review Update: In the recent updates, all vulnerabilities have been fixed.

ID	DESCRIPTION	SEVERITY	STATUS
NST-1	Staker Binding Front-Running via Direct `StakingOperators.stakeTo()`	Medium	Fixed
NST-2	`StakingOperators.stakeTo()` Does Not Account for Fee-on-Transfer Tokens Inflating Stake Accounting	Medium	Fixed
NST-3	Auto-Restake Can Re-Stake Rewards on a Fully-Exiting Operator Below `minStake`	Low	Fixed
NST-4	Token Migration Can Strand Residual Balances and Introduce Operational Risk on Reversal	Low	Fixed
NST-5	Uncapped Fee Rate	Low	Fixed
NST-6	No Operator Ownership Transfer or Factory Migration Path	Informational	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
NST-7	`Nodeoperator` Missing Token Recovery Function	Informational	Fixed
NST-8	`Nodeoperator` Does Not Check Jailed Node Status	Informational	Fixed
NST-9	Configuration Setters Can Create Incompatible Dependencies	Informational	Fixed
NST-10	Batch Harvesting Silently Hides Persistent Failures	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: https://github.com/NillionNetwork/blacklight-contracts/blob/main/src/NodeOperatorFactory.sol(8ce4597)

Repo: https://github.com/NillionNetwork/blacklight-contracts/blob/main/src/NodeOperator.sol(8ce4597)

Operational Considerations

1. **NodeOperator Isolation**: Each registered node address gets a dedicated NodeOperator contract deployed by the Factory at `addNode()` time. The NodeOperator is owned by the Factory and manages a single immutable `nodeAddress`. Users interact only with the Factory,

which routes operations to the appropriate NodeOperator. This provides per-node isolation — a failure in one NodeOperator does not affect others.

2. **Permanent User-to-Node Binding**: When a user first stakes, `_bindFreeNode()` pops the last free node (LIFO) and permanently maps the user to that node's operator. There is no unbind, rebind, or migration mechanism. If a user's bound node is slashed or performs poorly, the user cannot switch to a different node. However, users can fully unstake and withdraw their tokens — the binding is a routing mechanism, not a fund lockup. The LIFO selection means the most recently added nodes are bound first.
3. **Configuration Synchronization Requirement**: The Factory stores shared configuration (stakingOperators, rewardPolicy, token, minStake) and individual NodeOperators store their own copies. When Factory config changes, `syncOperatorConfig()` or `syncAllOperatorConfigs()` must be called to push updates. `_syncOperatorConfig()` sets stakingOperators, then rewardPolicy, then token (which validates consistency), then minStake. The sync is NOT automatic — admin must explicitly trigger it. There is a window between config change and sync where Factory and operators disagree.
4. **Sweep-All Balance Pattern**: `NodeOperator.stake()` uses `token.balanceOf(address(this))` rather than a passed amount parameter. This design intentionally handles fee-on-transfer tokens and ensures no tokens get permanently stuck. However, any tokens sent directly to a NodeOperator (accidentally or deliberately) will be swept into the next `stake()` call, benefiting the bound nodeUser.
5. **Fee Architecture**: NodeOperator collects fees on reward harvests: `withdrawFeeBps` (when rewards are withdrawn to user) and `restakeFeeBps` (when rewards are auto-restaked). Both capped at `MAX_FEE_BPS = 10000` (100%). Fees are sent to `owner()` (Factory). Factory accumulates fees and admin can withdraw via `withdrawFees()`. Default fees are 30% withdraw / 15% restake.
6. **Jailing Semantics**: StakingOperators enforces jail by blocking `requestUnstake()` during the jail period. `stakeTo()` does NOT check jail status — staking into jailed operators is permitted. Jailing is temporal with an `untilTimestamp` expiry. Jailed operators are removed from the active set via `_setActiveInSet()` and do not earn new rewards. After jail expires, operators can be reactivated and normal unstaking resumes.
7. **Batch Harvest with Silent Failures**: `harvestAllRewards(offset, limit)` uses `try ... catch {}` to silently swallow individual operator harvest failures. This prevents a single broken operator from blocking the entire batch. However, failed harvests produce no events or logs, making monitoring difficult.
8. **Assumed External Contract Security**: The system assumes the security of StakingOperators, RewardPolicy, HeartbeatManager, and other Blacklight protocol contracts. NodeOperator delegates critical operations (staking, unstaking, reward claiming) to these external contracts without additional validation beyond interface compliance.
9. **No Governance Takeover Assumed**: No governance takeover is assumed at any point in time.
10. **Token Standard Compliance**: The system assumes NIL token is a standard ERC20 with no transfer fees, rebasing, or balance-modifying hooks. RewardPolicy documentation explicitly states this assumption.
11. **Increasing `minStake` Retroactively Restricts Existing Stakers**: The Factory owner can update `minStake` via `setMinStake()`, which propagates to NodeOperators through `syncOperatorConfig()` / `syncAllOperatorConfigs()`. In `NodeOperator.requestUnstake()`, the check `if (remaining != 0 && remaining < minStake) revert BelowMinimumStake()` means that if `minStake` is raised above a user's current stake, they can only unstake to exactly zero — partial unstakes are blocked. Full exit is always available (the `remaining != 0` carve-out is intentional), but operators should be aware that a `minStake` increase retroactively constrains existing stakers' withdrawal flexibility. Additionally, there is a sync gap: after `setMinStake()` is called on the Factory, existing NodeOperators retain the old value until sync is explicitly triggered, so different operators may temporarily enforce different minimums.

Key Actors And Their Capabilities

1. **Users (NodeOperatorFactory)**: Can stake tokens via `NodeOperatorFactory.stake()`, which auto-deploys and auto-binds a dedicated `NodeOperator` instance on first stake (LIFO from free node pool). Users can request unstaking (`requestUnstake`), withdraw matured unstake tranches (`withdrawUnstaked`), claim/harvest rewards (`claimRewards`), and set their reward behavior between `AutoRestake` and `WithdrawToUser` (`setMyRewardBehavior`). Users interact only with the Factory; the Factory orchestrates operations through individual NodeOperator instances. User-to-node binding is permanent — there is no unbind or rebind mechanism.
2. **Factory Admin (NodeOperatorFactory.owner)**: Controls the entire registry configuration. Can add new nodes (`addNode`, `addNodes`), change core contract references (`setStakingOperators`, `setRewardPolicy`, `setToken`), adjust fees (`setDefaultModeFeeBps`, `setOperatorModeFeeBps`), update minimum stake (`setMinStake`), push config to operators (`syncOperatorConfig`, `syncAllOperatorConfigs`), and withdraw accumulated fees (`withdrawFees`). All admin functions are gated by `onlyOwner` with no timelock or multi-sig requirement at the contract level.
3. **NodeOperator (Per-Node Instance)**: Manages a single immutable `nodeAddress` and delegates staking/reward operations to `StakingOperators` and `RewardPolicy` on behalf of a bound user. Owned by the Factory. Exposes `stake()`, `requestUnstake()`, `withdrawUnstaked()`, `harvestRewards()`, `assignUser()`, and configuration setters — all gated by `onlyOwner` (Factory only). Uses a sweep-all balance pattern for staking (`token.balanceOf(address(this))`). Fee collection sends fees to `owner()` (Factory).
4. **Keepers/Bots (Permissionless Harvest)**: `NodeOperatorFactory.harvestRewards(operatorAddr)` and `harvestAllRewards()` are intentionally permissionless, allowing any address to trigger reward harvesting for any operator. This enables automated reward compounding and distribution without requiring user action.
5. **StakingOperators (External Dependency)**: Manages the actual stake accounting, operator registry, jailing, slashing, and unbonding tranches. NodeOperator contracts are permanently bound as the `operatorStaker` via the first-staker lock pattern. Not directly audited but analyzed at interface boundaries.
6. **RewardPolicy (External Dependency)**: Manages streaming-budget reward allocation with stake-weighted distribution. NodeOperator contracts claim rewards via `rewardPolicy.claim()`. Maintains accounting health checks and freeze mechanism. Not directly audited but analyzed at interface boundaries.

Findings

NST-1

Staker Binding Front-Running via Direct `StakingOperators.stakeTo()`

Medium · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `ad1fea06bbb1047f3810a5b6350e6fd3e48bf5f4`.
The client provided the following explanation:

> Our production deployment already includes the StakingOps smart contract (audited in a previous iteration). This makes it specially difficult to replace in production since there's stake in it. As such, to maintain it as similar as possible, we modify the NodeOperator smart contract deployment through the factory to use CREATE2 to produce predictable smart contract addresses based on the node address. That way, we can predict the smart contract address, approve the staker directly on the Staking smart contract and prevent the race condition. This doesn't ensure that it can be frontrun at the mempool level, but the new _addNodes function should prevent putting money at risk when adding the right node operator.

File(s) affected: `NodeOperatorFactory.sol`, `StakingOperators.sol`

Description: `StakingOperators.stakeTo()` is publicly callable with no access control beyond `nonReentrant` and `whenNotPaused`. When an operator has no bound `staker (operatorStaker[operator] == address(0))`, the first caller to `stakeTo()` permanently binds themselves as that operator's staker. The `approveStaker()` function exists to pre-approve a staker for unbound operators, but it requires the node address itself to be `msg.sender (approvedStaker[msg.sender] = staker)`. In the Factory flow, `_addNode()` deploys a `NodeOperator` contract that stakes on behalf of a pre-provisioned node address, but the node address is never involved as a caller, the Factory stakes *for* it, not *as* it. As a result, all Factory-managed nodes are deployed with `approvedStaker[node] == address(0)`, and `stakeTo()` permits any caller when no approval is set.

However, this means an attacker can front-run the Factory by calling `stakeTo(nodeAddress, minOperatorStake)` directly on `StakingOperators` after observing an `addNode()` transaction on-chain. Once the attacker binds themselves as the staker, the `NodeOperator` contract deployed by the Factory can no longer call `stakeTo()`, `requestUnstake()`, or `withdrawUnstaked()` for that node, all revert with `DifferentStaker()` because `operatorStaker[node]` is the attacker, not the `NodeOperator` contract. Notably, the existing `approveStaker()` function cannot be used as a fix in `_addNode()` because it keys on `msg.sender` as the operator: the Factory calling `approveStaker(nodeOperatorAddr)` would set `approvedStaker[Factory]`, not `approvedStaker[nodeAddress]`, making it ineffective.

This could lead to a griefing attack where the attacker only needs to front-run a single node, the last one in `_freeNodes` to block all Factory staking. Because `_bindFreeNode()` uses a LIFO pop `(_freeNodes[length - 1])`, every `Factory.stake()` call attempts to use the same poisoned last node, reverts with `DifferentStaker()`, and rolls back the entire transaction including the pop. The poisoned node remains at the top of the stack, making all nodes below it in the array permanently unreachable. The owner can temporarily restore staking by adding new nodes via `addNode()`, which push to the end of `_freeNodes` past the poisoned entry, but once those new nodes are consumed and the stack reaches the front-run node again, staking is blocked once more. The total cost to the attacker is a single `minOperatorStake` deposit (recoverable after the unstake delay). The admin can pause the contract to halt the attack, but this also halts all legitimate operations.

Exploit Scenario:

1. The admin calls `NodeOperatorFactory.addNode(nodeAddress)`, deploying a new `NodeOperator` at `operatorAddr`. No `StakingOperators.approveStaker()` call is made.
2. An attacker monitors the mempool and front-runs the transaction (or acts in the next block). The attacker calls `StakingOperators.stakeTo(nodeAddress, minStake)` directly, sending `minStake` tokens.
3. `operatorStaker[nodeAddress]` is permanently set to the attacker's address.
4. A legitimate user calls `NodeOperatorFactory.stake(amount)`. The `NodeOperator` calls `StakingOperators.stakeTo(nodeAddress, amount)`, which reverts with `DifferentStaker` because `operatorStaker[nodeAddress]` is the attacker, not the `NodeOperator` contract.
5. The node is permanently blocked from receiving stakes through the Factory. The attacker can repeat this for every newly added node, bricking the entire Factory.
6. The attacker's cost is `minStake` per node (recoverable via `StakingOperators.requestUnstake()` + `StakingOperators.withdrawUnstaked()` directly) plus gas. The attacker also captures any staking rewards via direct `RewardPolicy.claim()`.

Recommendation: Restrict `StakingOperators.stakeTo()` to only accept calls from whitelisted `NodeOperator` contracts. The `NodeOperatorFactory` should register each newly deployed `NodeOperator` address in a whitelist on `StakingOperators` as part of `addNode()`, and `stakeTo()` should revert if `msg.sender` is not whitelisted. This eliminates the race window entirely by preventing arbitrary callers from binding the staker slot.

NST-2

`StakingOperators.stakeTo()` Does Not Account for Fee-on-Transfer Tokens Inflating Stake Accounting

Medium · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `e95122b32a0069116b2e6ef2e11292cb712d7ec1`.
The client provided the following explanation:

> Though the NIL token does not have fee on transfer, we've implemented the fix, in case we redeploy the smart contract in the future.

File(s) affected: `src/StakingOperators.sol`, `src/NodeOperator.sol`

Description: `NodeOperator.stake()` explicitly uses the `balanceOf` pattern to determine the actual token amount available, with a comment stating "Use actual balance to support fee-on-transfer tokens." It then calls `stakingOperators.stakeTo(nodeAddress, available)` to forward this amount into `StakingOperators`. Inside `StakingOperators.stakeTo()`, the function calls `_stakingToken.safeTransferFrom(msg.sender, address(this), amount)` and then increments both `_operatorStake[operator]` and `_totalStaked` by the raw `amount` parameter.

However, if the staking token applies a transfer fee, the actual tokens received by `StakingOperators` are less than `amount`, while the accounting records the full `amount`. This creates a growing phantom balance where `_totalStaked` and `_operatorStake` exceed the contract's real token holdings. The inconsistency compounds with every stake operation and could lead to insolvency of the staking pool.

This could lead to last-withdrawer insolvency: when late operators attempt to withdraw via `withdrawUnstaked()`, the `safeTransfer` reverts because the contract holds fewer tokens than the recorded payout. Early withdrawers are unaffected, but late withdrawers permanently lose access to their stake.

Recommendation: Consider one solution:

- Modify `StakingOperators.stakeTo()` to measure actual tokens received using the `balanceOf` before/after pattern (consistent with `NodeOperator`), and use the actual received amount for `_operatorStake` and `_totalStaked` increments instead of the raw `amount` parameter.
- Modify the `NodeOperator` contract to not support fee on transfer tokens.
- Alternatively, if the behavior is desired consider documenting it.

NST-3

Auto-Restake Can Re-Stake Rewards on a Fully-Exiting Operator Below `minStake`

Low · Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `88e01e50ab84b427f14afc8bda6ebfdac011e405`.
> The client provided the following explanation:
>
> > When stake is 0, we automatically switch to rewards being sent to the user and prevent switching the mode back to AutoStake.

File(s) affected: `NodeOperator.sol`, `StakingOperators.sol`

Description: `NodeOperator.requestUnstake()` does not update `RewardBehavior` when the user fully exits (unstakes their entire balance). If the user's reward behavior is set to `AutoRestake` (the default, `NodeOperator.constructor()#L117`), any rewards that were earned before the exit but harvested afterwards will be automatically restaked onto the node via `_harvestIfPossible()#L318–321` — even though the user has already left. Since `harvestRewards()` is permissionless (`NodeOperatorFactory.harvestRewards()#L259`), a keeper or bot can trigger this at any time.

This silently partially reverses the user's full exit, leaving a small amount of stake on the node (potentially below `minStake`) and requiring the user to perform an additional unstake cycle to recover the funds. No permanent loss of funds occurs.

Exploit Scenario:

1. A user has 100 NIL staked with `RewardBehavior = AutoRestake` and `minStake = 50 NIL`.
2. The user earns 10 NIL in rewards (accrued in `RewardPolicy` from a prior period).
3. The user calls `requestUnstake(100)` to fully exit. Stake drops to 0, but the 10 NIL reward remains claimable.
4. A keeper calls `harvestRewards()`. The 10 NIL reward is claimed, a fee is deducted (e.g., 1.5 NIL), and the remaining 8.5 NIL is auto-restaked onto the node the user just exited.
5. The node now has 8.5 NIL staked (below `minStake`), and the user must unstake again to recover it.

Recommendation: Automatically switch `RewardBehavior` to `WithdrawToUser` when a full unstake is requested. In `NodeOperator.requestUnstake()`, when the remaining stake is zero, override the reward behavior so that any subsequent harvest sends rewards directly to the user rather than attempting to restake:

```
function requestUnstake(uint256 amount) external override nonReentrant onlyOwner {
    // ... existing validation ...
```

```
    stakingOperators.requestUnstake(nodeAddress, amount);

    // If fully exiting, force rewards to withdraw mode
    if (stakingOperators.stakeOf(nodeAddress) == 0) {
        _rewardBehavior = RewardBehavior.WithdrawToUser;
    }

    emit UnstakeRequested(nodeUser, amount, nodeAddress);
}
```

This captures the user's exit intent at the source, prevents both timing windows, and avoids the revert-on-harvest scenario entirely.

NST-4
Token Migration Can Strand Residual Balances and Introduce Operational Risk on Reversal

• Low Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `e35e5fd128f109f0836e3209f0f42f538d8e0d2c`.
The client provided the following explanation:

> Added token rescue functionality activated by the owner of the smart contract.

File(s) affected: `src/NodeOperator.sol`

Description: `NodeOperator.setToken()` allows the owner to replace the configured ERC-20 token, validating that the new address matches both `stakingOperators.stakingToken()` and `rewardPolicy.rewardToken()`. Since `StakingOperators._stakingToken` is immutable, a token change necessarily requires the `stakingOperators` address itself to change (pointing to a new `StakingOperators` contract with the new token). After the change, all runtime flows in `stake()`, `withdrawUnstaked()`, `_claimRewards()`, and `_harvestIfPossible()` use only the current `token` variable for balance accounting and transfers, and the contract stops interacting with the old token entirely.

However, there is no recovery path for residual balances of a previously configured token. While normal operations do not leave tokens in the NodeOperator between transactions (staking, withdrawing, and harvesting all sweep the full balance atomically), residual balances can exist from direct transfers, airdrops, or edge-case timing during migration. Once `token` is updated, these old-token balances become invisible to all contract functions. The only way to recover them is for the owner to change the `stakingOperators` and `token` back to the old values temporarily but this introduces significant operational complexity. During such a reversal window, users could call `Factory.stake()` which would transfer and stake the old token via the old `StakingOperators`. If the owner then changes the configuration forward again, those newly created positions in the old `StakingOperators` become orphaned, `NodeOperator` now points to the new `StakingOperators` and cannot call `requestUnstake()` or `withdrawUnstaked()` for those old positions.

This could lead to stranded token balances after migration that require error-prone manual recovery, and introduces operational risk if the owner reverts the token configuration, opening a window where new positions can be created in the old `StakingOperators` that become permanently inaccessible once the configuration changes forward again.

Recommendation:

- Add an explicit owner-gated rescue function for arbitrary ERC-20 balances that are not the currently active `token`, allowing recovery of residual balances without requiring a full configuration reversal.
- Alternatively, if the behavior is desired consider documenting it.

NST-5 Uncapped Fee Rate

• Low Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `41e4ce095c98d35a3c685bd9adf62f5deaf61b14`.
The client provided the following explanation:

> Set max fee rate to 50%

File(s) affected: `src/NodeOperator.sol`, `src/NodeOperatorFactory.sol`

Description: `NodeOperator.MAX_FEE_BPS` at L#54 is set to `10000` (100% in basis points). The setters `setModeFeeBps()` at L#146 and `setDefaultModeFeeBps()` in Factory at L#114 validate with `>` rather than `>=`, meaning a fee of exactly 10000 (100%) is accepted. When

`feeBpsToUse = 10000`, the fee calculation at L#299 yields `fee = (harvestableRewards * 10000) / 10000 = harvestableRewards`, and the net to the user is `harvestableRewards - fee = 0`. All harvested rewards go to the Factory owner as fees; the user receives nothing.

There is no timelock, governance gate, or per-user opt-in for fee changes. The admin can set fees to 100% at any time via `Factory.setDefaultModeFeeBps(10000, 10000)`, and the change takes effect immediately on the next harvest. Events are emitted (`ModeFeeBpsUpdated`), but no on-chain mechanism prevents or delays the change.

Recommendation: Cap fees at a reasonable maximum (e.g., 50%) and/or add a timelock, additionally, consider adding a timelock or governance delay for fee changes so users can react before new fees take effect.

NST-6
No Operator Ownership Transfer or Factory Migration Path

Informational · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `b66faf76ff92801d41cf2d165ee113caafcf3daf`.
The client provided the following explanation:

> Added migrateOperator functionality to enable factory migration, though functionality stays fixed locked for the same NodeOperator.

File(s) affected: `NodeOperatorFactory.sol`, `NodeOperator.sol`

Description: Each `NodeOperator` instance has its `Ownable.owner()` set to the `NodeOperatorFactory` address at deployment time. The factory exposes no function that calls `Ownable.transferOwnership()` on `NodeOperator` instances. If the factory contract needs to be replaced (e.g., critical bug, upgrade, or governance migration), there is no way to transfer control of existing operators to a new factory. Users would be forced to fully unstake, wait through the unbonding period, withdraw, and re-stake through the new factory, disrupting all active delegations.

Recommendation: Consider adding a guarded migration function to the factory:

```
function migrateOperator(address operatorAddr, address newOwner) external onlyOwner {
    require(newOwner != address(0), "invalid owner");
    NodeOperator(operatorAddr).transferOwnership(newOwner);
}
```

NST-7 `Nodeoperator` Missing Token Recovery Function

Informational · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `7ea93ea2cd8055a7a1950a6a44f3d75aa6daac03`.
The client provided the following explanation:

> Added a rescueTokens functionality for any token that is not the staking token.

File(s) affected: `NodeOperator.sol`

Description: No `recoverERC20()` function exists on `NodeOperator`. Non-NIL ERC-20 tokens accidentally sent to any `NodeOperator` contract are permanently locked. NIL tokens held by assigned operators are swept into the harvest flow by design. NIL tokens on unassigned operators can be rescued indirectly by the admin calling `NodeOperator.assignUser()` followed by a harvest cycle, but no such path exists for arbitrary ERC-20 tokens.

Recommendation: Add an `onlyOwner` recovery function with the signature `recoverERC20(address erc20, address to, uint256 amount)` that enforces `require(erc20 != address(token))` to prevent draining staked funds. A corresponding function in the `NodeOperatorFactory` contract is also needed to call this new `recoverERC20()`, since the factory is the owner of all `NodeOperator` contracts.

NST-8 `Nodeoperator` Does Not Check Jailed Node Status

Informational · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `abf5136d119e03a9661df857ac8b656a7b9bb12b`.
The client provided the following explanation:

> If the node is jailed, we prevent the node from allowing to stake any more amount. Though this should be addressed too at the Staking Operators level, we address it only at the NodeOperator level because of the re-deployment concerns on the Staking Operator contract that is already in production.

File(s) affected: `NodeOperator.sol`

Description: Neither `NodeOperator.stake()#L186` nor `NodeOperator._harvestIfPossible()` checks whether the node is jailed before staking. `StakingOperators.stakeTo()#L244` does not enforce a jail check either — deposits are silently accepted on jailed nodes. However, a jailed node is excluded from the active set (`StakingOperators._computeIsActive()#L194` returns false while `block.timestamp < _jailedUntil[operator]`), meaning it will not earn rewards. Additionally, `StakingOperators.requestUnstake()#L279` reverts with `OperatorJailed` while the node is jailed, preventing withdrawal until the jail period expires. A user who stakes into a jailed node will have their funds locked unproductively for the remaining jail duration with no way to exit early.

Recommendation: Add a jail check in `NodeOperator.stake()` and `NodeOperator._harvestIfPossible()` to prevent deposits on jailed nodes:

```
if (stakingOperators.isJailed(nodeAddress)) revert NodeJailed();
```

NST-9
Configuration Setters Can Create Incompatible Dependencies

Informational · Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `a2c8f3a5741f4a69e1b92a79450fb27fee96f3d2`.
The client provided the following explanation:

> We created a unified setDependencies function that allows setting the three dependencies, and checks they all match the expected token. This is only changed at the Factory level on purpose as it is the one that is externally facing. The NodeOperator receives information from the owner which has already been validated.

File(s) affected: `src/NodeOperatorFactory.sol`

Description: `setStakingOperators()` and `setRewardPolicy()` let the owner replace the factory's external dependency addresses at any time, while `setToken()` separately updates the token only after checking that the currently configured staking and reward dependencies both report the same asset. The intended flow is that newly added nodes and synchronized operators use one coherent tuple of staking contract, reward contract, and token.

However, `setStakingOperators()` and `setRewardPolicy()` do not validate compatibility against the currently configured token. This allows the factory to enter an internally inconsistent state where `stakingOperators`, `rewardPolicy`, and `token` no longer describe the same asset, even though the constructor originally enforced that invariant. Because downstream functions such as `_addNode()`, `withdrawFees()`, `pendingRewards()`, `stake()`, and `_syncOperatorConfig()` all trust these stored addresses, a misordered admin update can leave the contract configured around mismatched dependencies.

This could lead to broken node deployment, failed user staking flows, failed sync operations, and wrong token assumption. In practice, a simple privileged misconfiguration can disrupt the factory and every operator it provisions.

Recommendation:

- Add a single function that changes the 3 parameters together, validating the new configs against each other.
- Alternatively, if the behavior is desired consider documenting it.

NST-10 Batch Harvesting Silently Hides Persistent Failures

Informational · Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `4bacab992058a1473dcf48c6f69b52830e8ceca6`.
The client provided the following explanation:

> If a node fails to harvest a reward, it emits an event with the bytes as reason.

File(s) affected: `src/NodeOperatorFactory.sol`

Description: `harvestAllRewards(uint256 offset, uint256 limit)` iterates over `_allOperators` and wraps each external `harvestRewards()` call in a try/catch that deliberately swallows every failure. The intended flow is to keep batch harvesting live even if one operator is broken, so a single failure cannot revert the entire loop.

However, the function emits no event, tracks no failure count, and exposes no on-chain signal identifying which operators failed. As written, repeated failure of the same operator is indistinguishable from successful processing when viewed from this contract alone. Because harvesting is a core operational pathway and this function is public, the lack of any observable failure reporting turns real runtime issues into silent no-ops.

This could lead to persistent operator misconfigurations going undetected, with affected users unknowingly missing reward distributions indefinitely.

Recommendation:

- Emit an event when an operator harvest fails inside the batch loop, or return aggregate success and failure information so persistent faults can be detected and acted on. Keep the non-reverting batch behavior only if observability is added alongside it.
- Alternatively, if the behavior is desired consider documenting it.

Auditor Suggestions

S1 Residual Token Approval After `setToken` or `setStakingOperators` Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `a659b4b410acecf08b15ce7226562c64b00b7233`.
The client provided the following explanation:

> Before changing the staking contract, we change the allowance to 0 at the NodeOperator level which is the one generally dealing with the users funds.

File(s) affected: `NodeOperator.sol`

Description: `NodeOperator.setStakingOperators()#L125-128` replaces the staking contract reference without revoking the old contract's token allowance. If `forceApprove` was previously called (`NodeOperator.stake()#L197`, `NodeOperator._harvestIfPossible()#L319`) to grant the old `stakingOperators` address an allowance on `token`, that approval persists after the reference is updated. Similarly, `NodeOperator.setToken()#L137-143` replaces the token reference without revoking the old token's allowance to `stakingOperators`. In both cases, the stale approval remains on-chain. If a decommissioned staking contract is later compromised or contains an exploitable vulnerability, it could use the residual allowance to drain tokens from the `NodeOperator` contract.

Note that with the current `StakingOperators` contract implementation, `stakeTo()` always fully consumes the approved amount, so in practice there should not be remaining approvals left in the `NodeOperator` with the current intended contracts.

Recommendation: Revoke the old allowance before updating references in both functions:

```
// In setStakingOperators(), before updating the reference:
token.forceApprove(address(stakingOperators), 0);

// In setToken(), before updating the reference:
IERC20(oldToken).forceApprove(address(stakingOperators), 0);
```

S2 Missing Event Emissions on Factory Configuration Setters Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `8ad417c4e1240f74d2e1f32da3f1e21cd4eb462a`.
The client provided the following explanation:

> Added EventEmissions to Factory setters.

File(s) affected: `NodeOperatorFactory.sol`

Description: Six admin configuration functions emit no events: `NodeOperatorFactory.setStakingOperators()#L96`, `NodeOperatorFactory.setRewardPolicy()#L101`, `NodeOperatorFactory.setToken()#L106`, `NodeOperatorFactory.setDefaultModeFeeBps()#L113`, `NodeOperatorFactory.syncOperatorConfig()#L132`, and `NodeOperatorFactory.syncAllOperatorConfigs()#L139`. This is inconsistent with `NodeOperatorFactory.setMinStake()#L125`, which does emit an event following the standard pattern. The absence of events means off-chain monitoring systems, indexers, and governance dashboards cannot detect or record critical admin configuration changes, eliminating the audit trail for these operations.

Recommendation: Define and emit events for all six functions, following the pattern established by `NodeOperatorFactory.setMinStake()`. Each event should include both the old and new values to enable change tracking.

S3 `renounceOwnership()` Not Disabled on Factory

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `ce3e4e4c9ef7b69060c5a30304e494236704b934`.
The client provided the following explanation:

> Disabled renounceOwnership from NodeOperatorFactory.

File(s) affected: `NodeOperatorFactory.sol`

Description: `NodeOperatorFactory` inherits OpenZeppelin's `Ownable` but does not override `Ownable.renounceOwnership()`. If the owner calls this function (whether accidentally or through a compromised multisig action), ownership is irrevocably transferred to `address(0)`. All `onlyOwner` admin functions (`NodeOperatorFactory.setStakingOperators()`, `NodeOperatorFactory.setRewardPolicy()`, `NodeOperatorFactory.setToken()`, `NodeOperatorFactory.setDefaultModeFeeBps()`, `NodeOperatorFactory.setMinStake()`, `NodeOperatorFactory.syncOperatorConfig()`, `NodeOperatorFactory.syncAllOperatorConfigs()`) become permanently inaccessible, bricking configuration management for the entire staking system. Note that `NodeOperator` instances are owned by the factory, so their `renounceOwnership()` is not externally reachable.

Recommendation: Override `renounceOwnership()` with an unconditional revert:

```
function renounceOwnership() public pure override {
    revert("disabled");
}
```

S4 Factory Configuration Setters Lack Token-Compatibility Validation

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `a2c8f3a5741f4a69e1b92a79450fb27fee96f3d2`.
The client provided the following explanation:

> This is addressed in the same way as NST-17.

File(s) affected: `NodeOperatorFactory.sol`

Description: `NodeOperatorFactory.setStakingOperators()#L96` and `NodeOperatorFactory.setRewardPolicy()#L101` accept any non-zero address without verifying that the provided contract is compatible with the factory's configured token. This is inconsistent with `NodeOperatorFactory.setToken()#L106`, which does validate compatibility by calling `stakingToken()` and `rewardToken()` on the staking and reward contracts. If an admin sets an incompatible staking or reward contract address, all subsequent staking, unstaking, and reward operations will revert, causing a system-wide denial of service until the admin corrects the configuration.

Recommendation: Mirror the validation pattern used in `NodeOperatorFactory.setToken()`. In `setStakingOperators()`, call `stakingToken()` on the new address and verify the return value matches the factory's current `token`. Apply equivalent validation in `setRewardPolicy()`.

S5 `syncAllOperatorConfigs()` Omits Fee Rate Synchronization

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `71f3444e9ae553686c0b081ab213e459182abff6`.

The client provided the following explanation:

> Improved NatSpec to hopefully address the concerns raised in the suggestion.

File(s) affected: `NodeOperatorFactory.sol`

Description: `NodeOperatorFactory._syncOperatorConfig()#L149-155` pushes `token`, `stakingOperators`, `rewardPolicy`, and `minStake` to `NodeOperator` instances but intentionally does **not** propagate fee rates (`withdrawFeeBps`, `restakeFeeBps`). The test `test_setDefaultModeFeeBps_appliesToNewOperatorsOnly` explicitly asserts this behavior, confirming per-operator fee customization is a deliberate design choice. However, the NatSpec comment referencing "current config" is mildly misleading because it omits the fee exclusion.

Recommendation: Clarify in the NatSpec documentation for `NodeOperatorFactory._syncOperatorConfig()` and `NodeOperatorFactory.syncAllOperatorConfigs()` that fee rates are intentionally per-operator and excluded from synchronization.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

forge test

```
Ran 2 tests for test/NillionToken.t.sol:NillionTokenTest
[PASS] test_setMinter_onlyOwner_and_allowsMint() (gas: 91520)
[PASS] test_setMinter_rejectsZeroAddress() (gas: 11093)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.68ms (521.58µs CPU time)

Ran 6 tests for test/EmissionsController.t.sol:EmissionsControllerTest
[PASS] test_emissionForEpoch_bounds() (gas: 19216)
[PASS] test_ensureBridgeApproval_restoresMaxAllowance() (gas: 29180)
[PASS] test_globalCapExceeded_reverts() (gas: 1014576)
[PASS] test_mintAndBridge_enforcesEpochTiming() (gas: 177710)
[PASS] test_mintAndBridge_isPermissionless() (gas: 136020)
[PASS] test_setL2GasLimit_onlyOwner() (gas: 20705)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 3.24ms (906.75µs CPU time)

Ran 1 test for test/HeartbeatFlow.t.sol:HeartbeatFlowTest
[PASS] testFullHeartbeatFlow() (gas: 924942)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.15ms (527.17µs CPU time)

Ran 1 test for test/HeartbeatManagerConfig.t.sol:HeartbeatManagerConfigTest
[PASS] test_setConfig_onlyOwner_and_appliesToNewRounds() (gas: 1993742)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.01ms (896.04µs CPU time)

Ran 15 tests for test/StakingOperators.t.sol:StakingOperatorsTest
[PASS] test_adminSetters_onlyAdmin() (gas: 25456)
[PASS] test_adminSetters_updateValues() (gas: 1104976)
[PASS] test_adminSetters_validateBounds() (gas: 32492)
[PASS] test_fullUnstakeStillClearsBinding() (gas: 308326)
```

```
[PASS] test_jail_deactivates_and_requires_reactivate() (gas: 513045)
[PASS] test_registerOperator_revertsWhenActiveSetFull() (gas: 715027)
[PASS] test_register_requiresMinStake() (gas: 524191)
[PASS] test_requestUnstake_and_withdraw() (gas: 472871)
[PASS] test_requestUnstake_revertsWhileJailed() (gas: 420768)
[PASS] test_slash_active_and_unbonding() (gas: 469560)
[PASS] test_snapshot_and_stakeAt_checkpoints() (gas: 458858)
[PASS] test_stakeTo_allowsTopUpAfterMinIncrease() (gas: 334600)
[PASS] test_stakeTo_bindsStaker() (gas: 349065)
[PASS] test_stakeTo_requiresMinStakeOnInitialBind() (gas: 104595)
[PASS] test_stakeTo_revertsForUnauthorizedStakerWhenApproved() (gas: 341909)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 1.72ms (1.26ms CPU time)

Ran 1 test for test/HeartbeatFlow.t.sol:HeartbeatInvalidRewardFlowTest
[PASS] testInvalidThresholdRewardsInvalidVoters() (gas: 2026054)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.03ms (3.48ms CPU time)

Ran 17 tests for test/NodeOperator.t.sol:NodeOperatorTest
[PASS] test_assignAndStake() (gas: 223565)
[PASS] test_defaultRewardBehavior_isAutoRestake() (gas: 8397)
[PASS] test_directEntryPointsRevertForNonOwner() (gas: 49853)
[PASS] test_harvestRevertsIfRewardModulesUnset() (gas: 1380802)
[PASS] test_minStakeEnforced() (gas: 74304)
[PASS] test_nodeConfigured() (gas: 11286)
[PASS] test_partialUnstakeAndWithdraw_nodeStaysAssigned() (gas: 346375)
[PASS] test_partialUnstake_allowsFullUnstake() (gas: 285090)
[PASS] test_partialUnstake_allowsWhenRemainingAboveMinStake() (gas: 311723)
[PASS] test_partialUnstake_revertsWhenRemainderBelowMinStake() (gas: 231893)
[PASS] test_requestUnstakeAndWithdraw() (gas: 346492)
[PASS] test_rewardsHarvest() (gas: 285080)
[PASS] test_rewardsHarvest_forwardsFeesImmediatelyAndOnlyOnNewRewards() (gas: 349436)
[PASS] test_rewardsHarvest_restakeModeUsesRestakeFeeAndCompoundsStake() (gas: 329723)
[PASS] test_secondUserCannotBeAssignedWhileNodeInUse() (gas: 229304)
[PASS] test_setModeFeeBps_revertsAboveMaxCap() (gas: 15218)
[PASS] test_setModeFeeBps_setsModeFees() (gas: 56460)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 4.72ms (2.21ms CPU time)

Ran 7 tests for test/MediumFindingsPoC.t.sol:MediumFindingsPoC
[PASS] test_M1_directStakeToBypassesFactoryBinding() (gas: 451053)
[PASS] test_M1_griefingAllNodes() (gas: 731998)
[PASS] test_M2_100percentFeeExtractsAllRewards() (gas: 453138)
[PASS] test_M2_feeValidationAllows100Percent() (gas: 26857)
[PASS] test_M3_frontRunSlashWithWithdraw() (gas: 458161)
[PASS] test_M3_jailedOperatorCanWithdrawMatureTranches() (gas: 483910)
[PASS] test_M3_requestUnstakeBlockedForJailedOperator() (gas: 361392)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 1.60ms (778.21µs CPU time)

Ran 18 tests for test/NodeOperatorFactory.t.sol:NodeOperatorFactoryTest
[PASS] test_addNode_deploysAndConfiguresNodeOperator() (gas: 1640459)
[PASS] test_addNode_revertsForDuplicateNode() (gas: 1623874)
[PASS] test_addNode_revertsForNonOwner() (gas: 14537)
[PASS] test_claimRewards_restakeModeCompoundsAndTakesRestakeFee() (gas: 2037257)
[PASS] test_fullLifecycle_stakeUnstakeWithdraw() (gas: 1866714)
[PASS] test_harvestAllRewards_harvestsAllAssignedNodes() (gas: 3889309)
[PASS] test_isolatedRewards_twoUsersOnDifferentNodes() (gas: 3738139)
[PASS] test_pendingRewards_returnsCorrectAmount() (gas: 1900267)
[PASS] test_setDefaultModeFeeBps_appliesToNewOperatorsOnly() (gas: 3277786)
[PASS] test_setDefaultModeFeeBps_revertsAboveMaxCap() (gas: 16204)
[PASS] test_setMyRewardBehavior_revertsWhenUserUnbound() (gas: 19320)
[PASS] test_setOperatorModeFeeBps_ownerCanOverridePerOperator() (gas: 1666946)
[PASS] test_stake_autoBindsUserToFreeNode() (gas: 1899594)
[PASS] test_stake_revertsNoFreeNodeOperator() (gas: 1955717)
[PASS] test_stake_secondUserBindsToDifferentNode() (gas: 3687630)
[PASS] test_withdrawFees_ownerCanWithdrawHarvestedFees() (gas: 2063516)
[PASS] test_withdrawFees_revertsForNonOwner() (gas: 14356)
[PASS] test_withdrawUnstaked_preservesRewardBehavior() (gas: 1862490)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 5.76ms (4.17ms CPU time)

Ran 1 test for test/HeartbeatManagerCommitteeGrowth.t.sol:HeartbeatManagerCommitteeGrowthTest
[PASS] test_committeeSize_grows_on_escalation() (gas: 2153432)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.93ms (4.37ms CPU time)
```

```
Ran 6 tests for test/JailingPolicy.t.sol:JailingPolicyTest
[PASS] test_enforceJailFromMembers_jailsNonvoters_and_incorrectVoters() (gas: 1658022)
[PASS] test_enforceJailFromMembers_revertsOnRootMismatchOrUnsorted() (gas: 1201476)
[PASS] test_enforceJail_individualWithProof() (gas: 1384568)
[PASS] test_inconclusiveVote_notJailedOnInvalidOutcome() (gas: 1595608)
[PASS] test_inconclusiveVote_notJailedOnValidOutcome() (gas: 1595188)
[PASS] test_roundIsRecordedOnFinalize() (gas: 1174316)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 259.72ms (8.59ms CPU time)

Ran 9 tests for test/WeightedCommitteeSelector.t.sol:WeightedCommitteeSelectorTest
[PASS] test_largeCommitteeSelection() (gas: 73389827)
[PASS] test_selectCommittee_capsToMaxActiveOperators_whenPoolIsHuge() (gas: 565700906)
[PASS] test_selectCommittee_fallsBackToPrevrandao() (gas: 1649665)
[PASS] test_selectCommittee_respects_maxActiveOperators_pool() (gas: 6287073)
[PASS] test_selectCommittee_revertsWhenSnapshotPredatesStake() (gas: 419672)
[PASS] test_selectCommittee_unique_and_bounded() (gas: 6227340)
[PASS] test_setMaxActiveOperators_zeroResetsDefault() (gas: 19679)
[PASS] test_setMaxCommitteeSize_onlyAdmin_and_nonZero() (gas: 22279)
[PASS] test_setMinCommitteeVP_onlyAdmin_and_blocksSelection() (gas: 1357481)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 518.88ms (547.67ms CPU time)

Ran 8 tests for test/ProtocolConfig.t.sol:ProtocolConfigTest
[PASS] test_constructor_revertsOnInvalidBps() (gas: 63365)
[PASS] test_constructor_revertsOnZeroModules() (gas: 62121)
[PASS] test_setModules_onlyOwner() (gas: 1370850)
[PASS] test_setModules_rejectsEOA() (gas: 1146822)
[PASS] test_setNodeVersion_onlyOwner() (gas: 1042630)
[PASS] test_setParams_onlyOwner() (gas: 1013658)
[PASS] test_setParams_updatesValues() (gas: 1025131)
[PASS] test_setParams_validatesCommitteeCaps() (gas: 1013642)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 522.92ms (1.49ms CPU time)

Ran 10 tests for test/RewardPolicy.t.sol:RewardPolicyTest
[PASS] test_accrueWeights_revertsOnLengthMismatch() (gas: 210795)
[PASS] test_accrueWeights_revertsOnUnsortedRecipients() (gas: 212066)
[PASS] test_accrueWeights_uses_partial_unlock() (gas: 308535)
[PASS] test_claim_transfersAndClears() (gas: 304951)
[PASS] test_commitmentMismatch_afterInitialRevert() (gas: 333974)
[PASS] test_dustDistribution_allocatesOneWei() (gas: 291864)
[PASS] test_maxPayoutCap() (gas: 359240)
[PASS] test_setEpochDuration_onlyOwner_and_updatesValue() (gas: 63579)
[PASS] test_setMaxPayoutPerFinalize_onlyOwner() (gas: 39276)
[PASS] test_sync_underflow_updatesAccountedBalance_withoutFreeze() (gas: 138615)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 522.72ms (1.57ms CPU time)

Ran 9 tests for test/BlacklightSystem.t.sol:BlacklightSystemTest
[PASS] test_endToEnd_rewards_claim_by_staker() (gas: 15791858)
[PASS] test_endToEnd_rewards_large_shared_staker() (gas: 24233872)
[PASS] test_endToEnd_rewards_mixed_staker_and_operator_claims() (gas: 14473922)
[PASS] test_endToEnd_rewards_use_latest_staker_on_distribution() (gas: 13453484)
[PASS] test_endToEnd_valid_workflow_rewards_and_claim() (gas: 18675255)
[PASS] test_jailing_removesFromNextCommitteeSelection() (gas: 20920150)
[PASS] test_largeCommittee_200_members_finalizes_and_jailing_enforcement() (gas: 165450522)
[PASS] test_largeCommittee_weighted_rewards_track_stake_across_rounds() (gas: 73341054)
[PASS] test_veryLargeOperatorPool_reward_distribution_and_jailing() (gas: 558574353)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 14.36s (426.62ms CPU time)

Ran 18 tests for test/KeeperE2EPaths.t.sol:KeeperE2EPathsTest
[PASS] test_accounting_freeze_blocks_rewards() (gas: 12103800)
[PASS] test_bad_raw_htx_reverts() (gas: 12342352)
[PASS] test_duplicate_reward_processing_reverts() (gas: 13229252)
[PASS] test_escalation_then_finalizes() (gas: 13371292)
[PASS] test_exhausted_escalations_expires() (gas: 13212467)
[PASS] test_invalidThreshold_noBondRefund() (gas: 13093237)
[PASS] test_jailed_operator_inactive() (gas: 12337452)
[PASS] test_jailing_idempotent() (gas: 13107155)
[PASS] test_member_proof_invalid_reverts() (gas: 12728617)
[PASS] test_min_committee_vp_reverts() (gas: 12386591)
[PASS] test_min_stake_deactivates_operator() (gas: 12298937)
[PASS] test_noQuorum_expires() (gas: 12844873)
[PASS] test_reward_budget_empty_reverts() (gas: 12985259)
[PASS] test_reward_weight_mismatch_reverts() (gas: 13096626)
```

```
[PASS] test_slashing_callback_failure_keeps_notified_false() (gas: 12534956)
[PASS] test_slashing_gas_limit_too_low_fails_callback() (gas: 12453759)
[PASS] test_snapshot_edge_reverts() (gas: 12188102)
[PASS] test_validThreshold_rewards_jailing_and_claim() (gas: 13236284)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 14.36s (28.24ms CPU time)

Ran 39 tests for test/HeartbeatManager.t.sol:HeartbeatManagerTest
[PASS] test_abandonRewardDistribution_blocksPayout_and_is_ownerOnly() (gas: 1209770)
[PASS] test_committeeRoot_matchesOffchainMerkleComputation() (gas: 754932)
[PASS] test_distributeRewards_aggregatesNonAdjacentStakers() (gas: 14181634)
[PASS] test_distributeRewards_creditsStakerRecipients() (gas: 13751394)
[PASS] test_distributeRewards_fallsBackToOperatorWhenStakerCleared() (gas: 1742655)
[PASS] test_distributeRewards_revertsOnInconclusiveOutcome() (gas: 1333870)
[PASS] test_distributeRewards_revertsOnInsufficientBudget() (gas: 1270740)
[PASS] test_distributeRewards_revertsOnInvalidVoterInList() (gas: 1391657)
[PASS] test_distributeRewards_revertsOnUnsortedVoters() (gas: 1384255)
[PASS] test_distributeRewards_revertsOnWeightMismatch_invalidOutcome() (gas: 1186849)
[PASS] test_distributeRewards_revertsOnWeightMismatch_validOutcome() (gas: 1386627)
[PASS] test_distributeRewards_revertsWhenAccountingFrozen() (gas: 1452265)
[PASS] test_distributeRewards_sortsStakerRecipients() (gas: 13801071)
[PASS] test_doubleVote_reverts() (gas: 850101)
[PASS] test_escalateOrExpire_beforeDeadline_reverts() (gas: 736720)
[PASS] test_escalateOrExpire_inconclusive_startsNewRound_and_then_expires() (gas: 1470740)
[PASS] test_escalateOrExpire_revertsAtDeadline() (gas: 1243966)
[PASS] test_escalateOrExpire_revertsOnRawHashMismatch() (gas: 749606)
[PASS] test_escalateOrExpire_revertsOnRawHashMismatch_afterEscalation() (gas: 1254125)
[PASS] test_finalize_invalidThreshold_updatesStatus() (gas: 1176859)
[PASS] test_finalize_validThreshold_updatesStatus() (gas: 1176207)
[PASS] test_minOperatorStake_change_after_votes_does_not_break_distribution() (gas: 1646978)
[PASS] test_moduleUpgrade_doesNotAffectExistingRoundRewardAddress() (gas: 3033411)
[PASS] test_moduleUpgrade_doesNotAffectExistingRoundSelector() (gas: 1692692)
[PASS] test_moduleUpgrade_doesNotAffectExistingRoundStakingOps() (gas: 5068778)
[PASS] test_pause_blocks_submitHeartbeat_and_votes() (gas: 758968)
[PASS] test_retrySlashing_retriesAfterFailure() (gas: 1283231)
[PASS] test_retrySlashing_revertsWhenNotFinalized() (gas: 734429)
[PASS] test_setSlashingGasLimit_ownerOnly_and_nonZero() (gas: 27795)
[PASS] test_submitHeartbeat_idempotent_noSecondRoundStarted() (gas: 733184)
[PASS] test_submitHeartbeat_revertsForInvalidSnapshotId() (gas: 247143)
[PASS] test_submitHeartbeat_revertsForUnauthorizedSubmitter() (gas: 251770)
[PASS] test_submitHeartbeat_startsRoundAndSetsPending() (gas: 756553)
[PASS] test_submitVerdict_revertsAfterBatchedVote() (gas: 877775)
[PASS] test_submitVerdict_revertsAfterDeadline() (gas: 767189)
[PASS] test_submitVerdict_revertsForNonMember() (gas: 736699)
[PASS] test_submitVerdictsBatched_hardLimit500() (gas: 580948)
[PASS] test_submitVerdictsBatched_respects_configMaxBatch() (gas: 861535)
[PASS] test_submitVerdictsBatched_signature() (gas: 887888)
Suite result: ok. 39 passed; 0 failed; 0 skipped; finished in 14.36s (49.13ms CPU time)

Ran 3 tests for test/invariants/HeartbeatManagerInvariants.t.sol:HeartbeatManagerInvariants
[PASS] invariant_committeeTotalStakeMatchesSumStakeAtSnapshot() (runs: 193, calls: 96500, reverts: 0)
[PASS] invariant_roundAccountingSums() (runs: 196, calls: 98000, reverts: 0)
[PASS] invariant_voteWeightsMatchSnapshotStake() (runs: 194, calls: 97000, reverts: 0)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 16.45s (35.18s CPU time)

Ran 4 tests for test/invariants/StakingInvariants.t.sol:StakingInvariants
[PASS] invariant_activeSetHasNoDuplicatesAndIsConsistent() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_operatorStakerZeroImpliesNoStakeAndNoUnbonding() (runs: 256, calls: 128000, reverts: 0)
[PASS] invariant_totalStakedEqualsSumOfActiveAndUnbondingAcrossTrackedOperators() (runs: 256, calls:
128000, reverts: 0)
[PASS] invariant_totalStakedMatchesTokenBalance() (runs: 256, calls: 128000, reverts: 0)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 28.16s (62.71s CPU time)

Ran 19 test suites in 28.16s (89.57s CPU time): 175 tests passed, 0 failed, 0 skipped (175 total tests)
```

Changelog

- 2026-03-23 - Initial report
- 2026-03-27 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 – Quantstamp, Inc.